

- a. The two Generative AIs I used for this assignment are Gemini and ChatGPT.
- b. I accessed both AIs from their respective sites: gemini.google.com and chatgpt.com
- c. The prompt I used for both generated code is as following:

- This program is written in C++ using objects, not functions.
- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.
- Test File format:

Command	Description
EMAIL <sender category>, <subject line>, <date>	The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows: • Boss – read first • Subordinate – read next • Peer – read next • ImportantPerson – read next • OtherPerson – read last If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint. EMAIL is followed by space. The rest of the fields are delimited. Assume <sender category> is one of the five strings listed above.

	Assume <subject line> is a string which may contain spaces, but not commas Assume <date> is in the format: MM-DD-YYYY
NEXT	Next email for the CEO to read. Display the information on the terminal in the following format: Sender: Subject: Date:
READ	CEO has read email and dealt with it
COUNT	Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss
Subject: Never Mind
Date: 01-03-2025

Next email:

Sender: Boss
Subject: Important
Date: 12-20-2024

There are 4 emails to read

d. Below are the two generated codes:

Gemini's Code:

```
#include <iostream>
#include <string>
#include <fstream>
#include <sstream>

// Forward declarations & helper functions
class Date {
public:
    int month, day, year;

    Date() : month(0), day(0), year(0) {}
    Date(const std::string& dateStr) {
        // Expected format: MM-DD-YYYY
        if (dateStr.length() >= 10) {
            month = std::stoi(dateStr.substr(0, 2));
            day = std::stoi(dateStr.substr(3, 2));
            year = std::stoi(dateStr.substr(6, 4));
        } else {
            month = day = year = 0;
        }
    }

    // Returns true if this date is newer (greater) than other
    bool operator>(const Date& other) const {
        if (year != other.year) return year > other.year;
        if (month != other.month) return month > other.month;
        return day > other.day;
    }

    bool operator==(const Date& other) const {
        return year == other.year && month == other.month && day == other.day;
    }
};
```

```

}
};

class Email {
private:
    std::string sender;
    std::string subject;
    std::string dateStr;
    Date date;
    int categoryPriority; // Higher integer = higher priority

    int assignPriority(const std::string& category) {
        if (category == "Boss") return 5;
        if (category == "Subordinate") return 4;
        if (category == "Peer") return 3;
        if (category == "ImportantPerson") return 2;
        if (category == "OtherPerson") return 1;
        return 0;
    }

public:
    Email() : sender(""), subject(""), dateStr(""), categoryPriority(0) {}
    Email(const std::string& snd, const std::string& sbj, const std::string& dt)
        : sender(snd), subject(sbj), dateStr(dt), date(dt) {
        categoryPriority = assignPriority(snd);
    }

    std::string getSender() const { return sender; }
    std::string getSubject() const { return subject; }
    std::string getDateStr() const { return dateStr; }

    // Overload comparison operators for MaxHeap priority ordering
    // Priority Rule 1: Sender Category (Boss > Subordinate > Peer > ImportantPerson > OtherPerson)
    // Priority Rule 2: Newer dates come first
    bool operator>(const Email& other) const {
        if (categoryPriority != other.categoryPriority) {
            return categoryPriority > other.categoryPriority;
        }
        return date > other.date; // Newest email first
    }

    bool operator<(const Email& other) const {
        if (categoryPriority != other.categoryPriority) {
            return categoryPriority < other.categoryPriority;
        }
        return !(date > other.date) && !(date == other.date);
    }
};

// Node structure for List-Based Tree Representation
struct HeapNode {
    Email data;

```

```

HeapNode* left;
HeapNode* right;
HeapNode* parent;

HeapNode(const Email& email)
: data(email), left(nullptr), right(nullptr), parent(nullptr) {}
};

// Helper Queue implementation for Level-Order Traversal (List-based)
struct QueueNode {
HeapNode* treeNode;
QueueNode* next;
QueueNode(HeapNode* node) : treeNode(node), next(nullptr) {}
};

class NodeQueue {
private:
QueueNode* frontNode;
QueueNode* rearNode;
public:
NodeQueue() : frontNode(nullptr), rearNode(nullptr) {}
~NodeQueue() {
while (!isEmpty()) {
dequeue();
}
}

bool isEmpty() const { return frontNode == nullptr; }

void enqueue(HeapNode* node) {
QueueNode* temp = new QueueNode(node);
if (rearNode == nullptr) {
frontNode = rearNode = temp;
return;
}
rearNode->next = temp;
rearNode = temp;
}

HeapNode* dequeue() {
if (isEmpty()) return nullptr;
QueueNode* temp = frontNode;
HeapNode* treeNode = temp->treeNode;
frontNode = frontNode->next;
if (frontNode == nullptr) rearNode = nullptr;
delete temp;
return treeNode;
}
};

// Custom List-Based MaxHeap Implementation
class MaxHeap {

```

```

private:
HeapNode* root;
int nodeCount;

void swapData(HeapNode* a, HeapNode* b) {
    Email temp = a->data;
    a->data = b->data;
    b->data = temp;
}

void siftUp(HeapNode* node) {
    while (node->parent != nullptr && node->data > node->parent->data) {
        swapData(node, node->parent);
        node = node->parent;
    }
}

void siftDown(HeapNode* node) {
    while (node != nullptr) {
        HeapNode* largest = node;
        if (node->left != nullptr && node->left->data > largest->data) {
            largest = node->left;
        }
        if (node->right != nullptr && node->right->data > largest->data) {
            largest = node->right;
        }
        if (largest != node) {
            swapData(node, largest);
            node = largest;
        } else {
            break;
        }
    }
}

HeapNode* getLastNode() {
    if (root == nullptr) return nullptr;
    NodeQueue q;
    q.enqueue(root);
    HeapNode* current = nullptr;
    while (!q.isEmpty()) {
        current = q.dequeue();
        if (current->left) q.enqueue(current->left);
        if (current->right) q.enqueue(current->right);
    }
    return current;
}

HeapNode* getInsertionParent() {
    NodeQueue q;
    q.enqueue(root);
    while (!q.isEmpty()) {
        HeapNode* current = q.dequeue();
    }
}

```

```
if (current->left == nullptr || current->right == nullptr) {  
    return current;  
}  
q.enqueue(current->left);  
q.enqueue(current->right);  
}  
return nullptr;  
}
```

```
void destroyTree(HeapNode* node) {  
    if (node == nullptr) return;  
    destroyTree(node->left);  
    destroyTree(node->right);  
    delete node;  
}
```

```
public:  
MaxHeap() : root(nullptr), nodeCount(0) {}  
~MaxHeap() { destroyTree(root); }
```

```
int size() const { return nodeCount; }  
bool isEmpty() const { return nodeCount == 0; }
```

```
void insert(const Email& email) {  
    HeapNode* newNode = new HeapNode(email);  
    nodeCount++;  
}
```

```
if (root == nullptr) {  
    root = newNode;  
    return;  
}
```

```
HeapNode* parent = getInsertionParent();  
newNode->parent = parent;  
if (parent->left == nullptr) {  
    parent->left = newNode;  
} else {  
    parent->right = newNode;  
}
```

```
siftUp(newNode);  
}
```

```
Email getMax() const {  
    if (isEmpty()) return Email();  
    return root->data;  
}
```

```
void removeMax() {  
    if (isEmpty()) return;
```

```

if (nodeCount == 1) {
delete root;
root = nullptr;
nodeCount = 0;
return;
}

HeapNode* lastNode = getLastNode();
root->data = lastNode->data;

HeapNode* parent = lastNode->parent;
if (parent->left == lastNode) {
parent->left = nullptr;
} else {
parent->right = nullptr;
}

delete lastNode;
nodeCount--;

siftDown(root);
}
};

// Priority Queue Wrapper Object
class PriorityQueue {
private:
MaxHeap heap;

public:
void enqueue(const Email& email) { heap.insert(email); }
void dequeue() { heap.removeMax(); }
Email peek() const { return heap.getMax(); }
int size() const { return heap.size(); }
bool isEmpty() const { return heap.isEmpty(); }
};

// CEO Email Manager Object to execute commands
class CEOEmailManager {
private:
PriorityQueue emailQueue;

std::string trim(const std::string& str) {
size_t first = str.find_first_not_of(" \t\r\n");
if (first == std::string::npos) return "";
size_t last = str.find_last_not_of(" \t\r\n");
return str.substr(first, (last - first + 1));
}

public:
void processCommand(const std::string& line) {

```

```

std::string trimmedLine = trim(line);
if (trimmedLine.empty()) return;

if (trimmedLine.rfind("EMAIL", 0) == 0) {
    std::string payload = trimmedLine.substr(5);
    std::stringstream ss(payload);
    std::string category, subject, date;

    std::getline(ss, category, ',');
    std::getline(ss, subject, ',');
    std::getline(ss, date, ',');

    Email email(trim(category), trim(subject), trim(date));
    emailQueue.enqueue(email);
}
else if (trimmedLine == "COUNT") {
    std::cout << "There are " << emailQueue.size() << " emails to read." << std::endl << std::endl;
}
else if (trimmedLine == "NEXT") {
    if (!emailQueue.isEmpty()) {
        Email top = emailQueue.peek();
        std::cout << "Next email:" << std::endl;
        std::cout << "Sender: " << top.getSender() << std::endl;
        std::cout << "Subject: " << top.getSubject() << std::endl;
        std::cout << "Date: " << top.getDateStr() << std::endl << std::endl;
    } else {
        std::cout << "No emails to display." << std::endl << std::endl;
    }
}
else if (trimmedLine == "READ") {
    if (!emailQueue.isEmpty()) {
        emailQueue.dequeue();
    }
}
};

int main() {
    CEOEmailManager manager;

    // Direct input stream simulation matching sample command stream
    std::string commands[] = {
        "EMAIL Peer,Can you help me on this?,12-01-2024",
        "EMAIL OtherPerson,Try our product,12-19-2024",
        "EMAIL Boss,Important,12-20-2024",
        "EMAIL Subordinate,How do I handle this?,12-25-2024",
        "EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024",
        "EMAIL Boss,Never Mind,01-03-2025",
        "COUNT",
        "NEXT",
        "READ",
        "NEXT",
        "READ",
    };
}

```

```

"COUNT"
};

for (const auto& cmd : commands) {
manager.processCommand(cmd);
}

return 0;
}

```

ChatGPT's code:

```

#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>

using namespace std;

class Email {
private:
string sender;
string subject;
string date;

int priority;
int month;
int day;
int year;

public:
Email(string sender, string subject, string date) {
this->sender = sender;
this->subject = subject;
this->date = date;

// Assign sender priority
if (sender == "Boss") {
priority = 5;
}
else if (sender == "Subordinate") {
priority = 4;
}
else if (sender == "Peer") {

```

```

priority = 3;
}
else if (sender == "ImportantPerson") {
priority = 2;
}
else {
priority = 1;
}

// Extract date: MM-DD-YYYY
month = stoi(date.substr(0, 2));
day = stoi(date.substr(3, 2));
year = stoi(date.substr(6, 4));
}

string getSender() {
return sender;
}

string getSubject() {
return subject;
}

string getDate() {
return date;
}

// Returns true if THIS email has higher priority
// than the other email.
bool higherPriorityThan(const Email& other) const {
// First compare sender category
if (priority != other.priority) {
return priority > other.priority;
}

// Same sender category: newest email first
if (year != other.year) {
return year > other.year;
}

if (month != other.month) {
return month > other.month;
}

return day > other.day;
}

void display() {
cout << "Sender: " << sender << endl;
cout << "Subject: " << subject << endl;
cout << "Date: " << date << endl;
}

```

```

}
};

class MaxHeap {
private:
vector<Email> heap;

public:
void add(const Email& email) {
heap.push_back(email);

int index = heap.size() - 1;

// Bubble the new email upward
while (index > 0) {
int parent = (index - 1) / 2;

if (heap[index].higherPriorityThan(heap[parent])) {
swap(heap[index], heap[parent]);
index = parent;
}
else {
break;
}
}

void next() {
if (heap.empty()) {
cout << "There are no emails to read." << endl;
return;
}

cout << "Next email:" << endl;
heap[0].display();
}

void read() {
if (heap.empty()) {
return;
}

// If only one email exists, remove it
if (heap.size() == 1) {
heap.pop_back();
return;
}

// Move last email to root
heap[0] = heap.back();

```

```

heap.pop_back();

int index = 0;

// Bubble downward
while (true) {
    int left = 2 * index + 1;
    int right = 2 * index + 2;
    int largest = index;

    if (left < heap.size() &&
        heap[left].higherPriorityThan(heap[largest])) {
        largest = left;
    }

    if (right < heap.size() &&
        heap[right].higherPriorityThan(heap[largest])) {
        largest = right;
    }

    if (largest == index) {
        break;
    }

    swap(heap[index], heap[largest]);
    index = largest;
}

void count() {
    cout << "There are " << heap.size() << " emails to read." << endl;
}
};

int main(int argc, char* argv[]) {

    if (argc < 2) {
        cout << "Please provide an input file." << endl;
        return 1;
    }

    ifstream inputFile(argv[1]);

    if (!inputFile.is_open()) {
        cout << "Unable to open file." << endl;
        return 1;
    }

    MaxHeap inbox;

```

```
string line;

while (getline(inputFile, line)) {

    if (line.substr(0, 6) == "EMAIL ") {

        // Remove "EMAIL "
        string emailInfo = line.substr(6);

        // Find the commas
        int firstComma = emailInfo.find(',');
        int secondComma = emailInfo.find(',', firstComma + 1);

        string sender =
            emailInfo.substr(0, firstComma);

        string subject =
            emailInfo.substr(firstComma + 1,
                secondComma - firstComma - 1);

        string date =
            emailInfo.substr(secondComma + 1);

        Email email(sender, subject, date);

        inbox.add(email);
    }

    else if (line == "NEXT") {
        inbox.next();
        cout << endl;
    }

    else if (line == "READ") {
        inbox.read();
    }

    else if (line == "COUNT") {
        inbox.count();
        cout << endl;
    }
}

inputFile.close();

return 0;
}
```

- e. In terms of correctness, a major issue is that Gemini's code cannot read a file. It only performs sample commands. On the other hand, ChatGPT takes a file name and processes its commands. Both original codes fail to check incorrect dates before using them, letting dates like 13-00-2026 go through without errors. ChatGPT also uses a vector, which needed to be changed to meet the list-based requirement.

Considering execution time, Gemini has a higher time complexity because it searches through its nodes to find where to insert or remove an email. This makes both operations $O(n)$. ChatGPT avoids these searches by using vector indices. Because it does not have to traverse through items it ends up being $O(\log n)$ instead.

For space complexity, both codes store emails in $O(n)$ space. However, Gemini requires a left, right, and parent pointer for every email node. It also uses a temporary queue that can take $O(n)$ space. ChatGPT avoids this queue and uses a fixed number of local variables for its heap operations. However, its vector capacity may be higher than its actual size.

Finally, in terms of maintainability, Gemini's code is harder for me to follow because it is longer and has more methods for queue traversal and pointer management. ChatGPT's code is shorter and straightforward, but printing inside `next()` and `count()` makes the heap less reusable. Both codes have some comments, but need more detailed comments for the assignment.

- f. The code I used for the basis of this assignment is ChatGPT's code due to having already better optimization for space and time complexity.
- g. I improved correctness by checking that dates are correctly formatted and valid for their month and year. Invalid dates now show an error and are skipped. I also changed the vector into linked nodes to meet the list-based requirement.

For execution time, my linked heap follows a path to the needed node instead of searching through the nodes like Gemini. This keeps insertion and removal at $O(\log n)$. I also replaced `std::endl` with `\n` to make it execute faster.

For space, I used references for the Email constructor and get functions to avoid unnecessary string copies. The linked heap still stores emails in $O(n)$ space and adds pointers, but does not need Gemini's temporary queue. I also added cleanup to delete the nodes when finished.

To improve maintainability, I added inline comments to the code and methods. I also moved printing for `next` and `count` into `main` so that the max heap has more reusability.